30.11.21

OPENCAMPUS.sh

Einführung in Data Science und maschinelles Lernen

NEURONALE NETZE



- Wiederholung Neuronale Netze (NN)
- Modellgütekriterien
- Hyperparameter in NN
- Frameworks zur Implementierung von NN
- Implementierung eines NN mit TensorFlow und Python

waterkant

CODING. WATERKANT 2022

7. bis 10. Juni auf dem Waterkant Festival Gelände

AI and Art
(Vladimir Alexeev, OpenAI Embassador; Michael Haas, Freelance Digital Artist)

KI.EL Racing Challenge
(Jake Petersen, opencampus.sh)

Detection of Mobile Genetic Elements
(Yiqing Wang and Dustin Hanke, University of Kiel)

Identifying Correct Arguments in Open Text Answers
(Christian Mayer and Georg Gorshid, University of Mannheim)

Identifying Geo-Locations in Historical Documents
(Simon Van der Wulp and Juan Veliz, north.io)

Predicting Stock Price Dynamics
(Jonas Mielck, University of Kiel and stackOcean GmbH)

Automated Feedback for Students' Argumentative Essays
(Thorben Jansen and Nils-Jonathan Schaller, IPN Kiel)

Enabling Researchers to Automatically Code their Text Responses from Assessments
(Fabian Zehner und Nico Andersen, DIPF Frankfurt)

Improving Lung Imaging with AI
(Claus Glüer, Intelligent Imaging Lab at the University of Kiel)



7. Juni auf dem Waterkant Festival Gelände

09:00 Welcome to Coding.Waterkant

09:15 Project Pitches

10:00 Common Breakfast

11:00 Introduction workshops on the projects that are open for participation

12:00 Meet the Expert: Q&A with Ralf Krestel from the ZBW/University of Kiel

13:00 Lunch Break

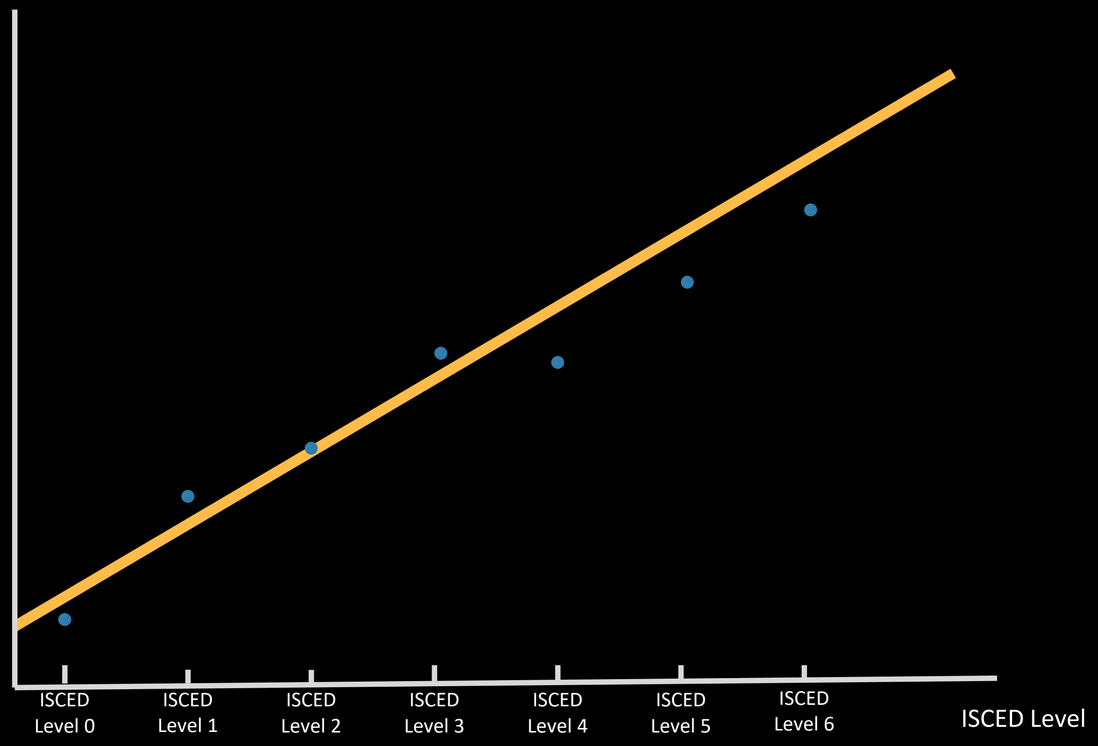
15:00 Thies Schönfeldt (meteolytix GmbH): The Do's and Don'ts in Time Series Prediction

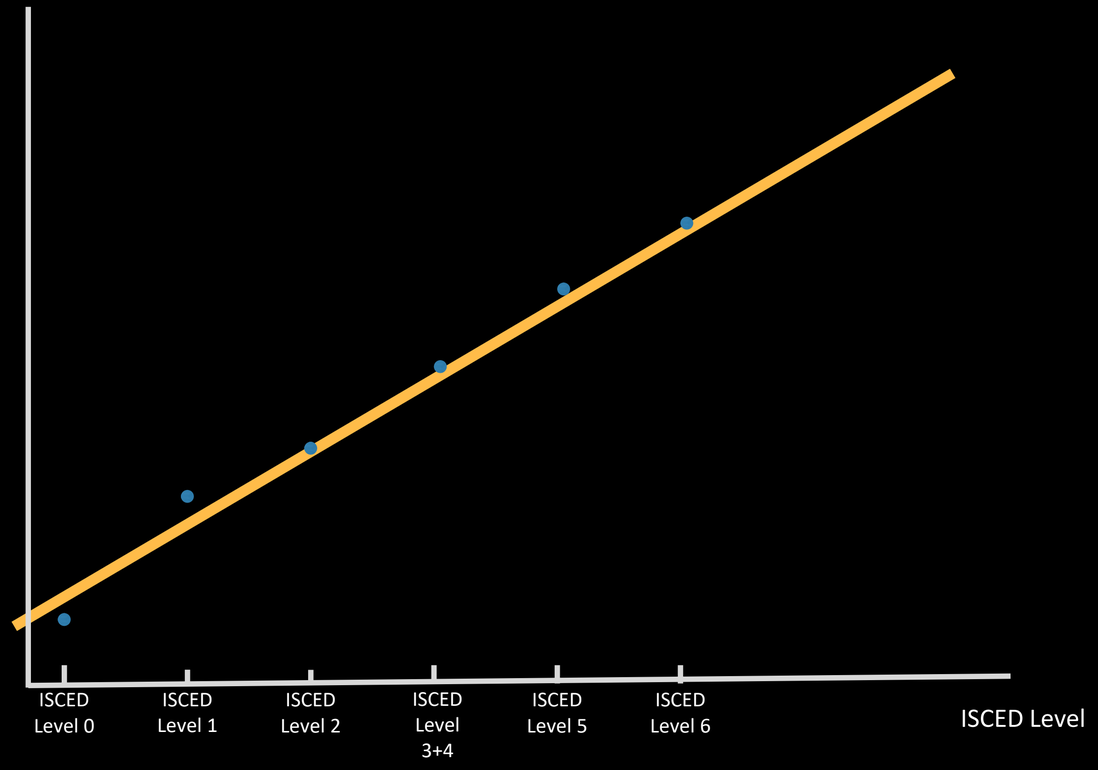
16:00 Jan Deller (LMU München/opencampus.sh): Introduction to TensorBoard

18:00 Kursstart: Zeitreihenanalysen

19:30 BBQ

RELEVANZ VON KATEGORISIERUNGEN





Wahl eines Prognosemodells

Teilung der Daten in Trainings- (70%), Validierungs- (20%) und Testdatensatz (10%)

Optimierung der Modellparameter anhand des Trainingsdatensatzes

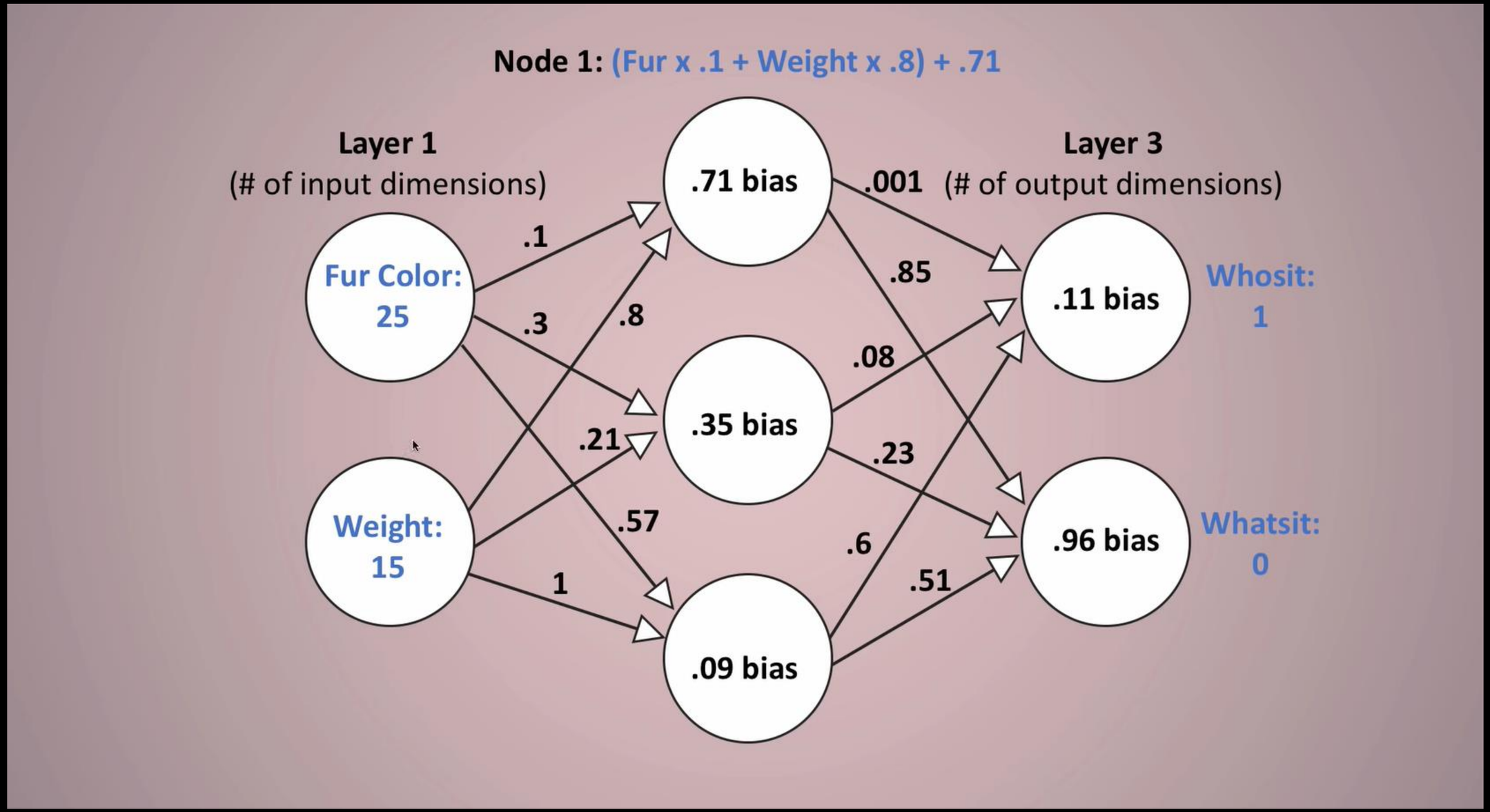
Optimierung der Hyperparameter anhand des Validierungsdatensatzes

Erweiterung/Verbesserung des Datensatzes

Überprüfung der Modellqualität anhand des Testdatensatzes

Verändern der Hyperparameter (modellzentrierte Optimierung

Verändern der Input- Variablen (datenzentrierte Optimierung)



LearnCode.academy. (2018, June 11). *Neural Networks Explained—Machine Learning Tutorial for Beginners*. https://www.youtube.com/watch?v=GvQwE2OhL8I

NEURONALE NETZE

Input Layer

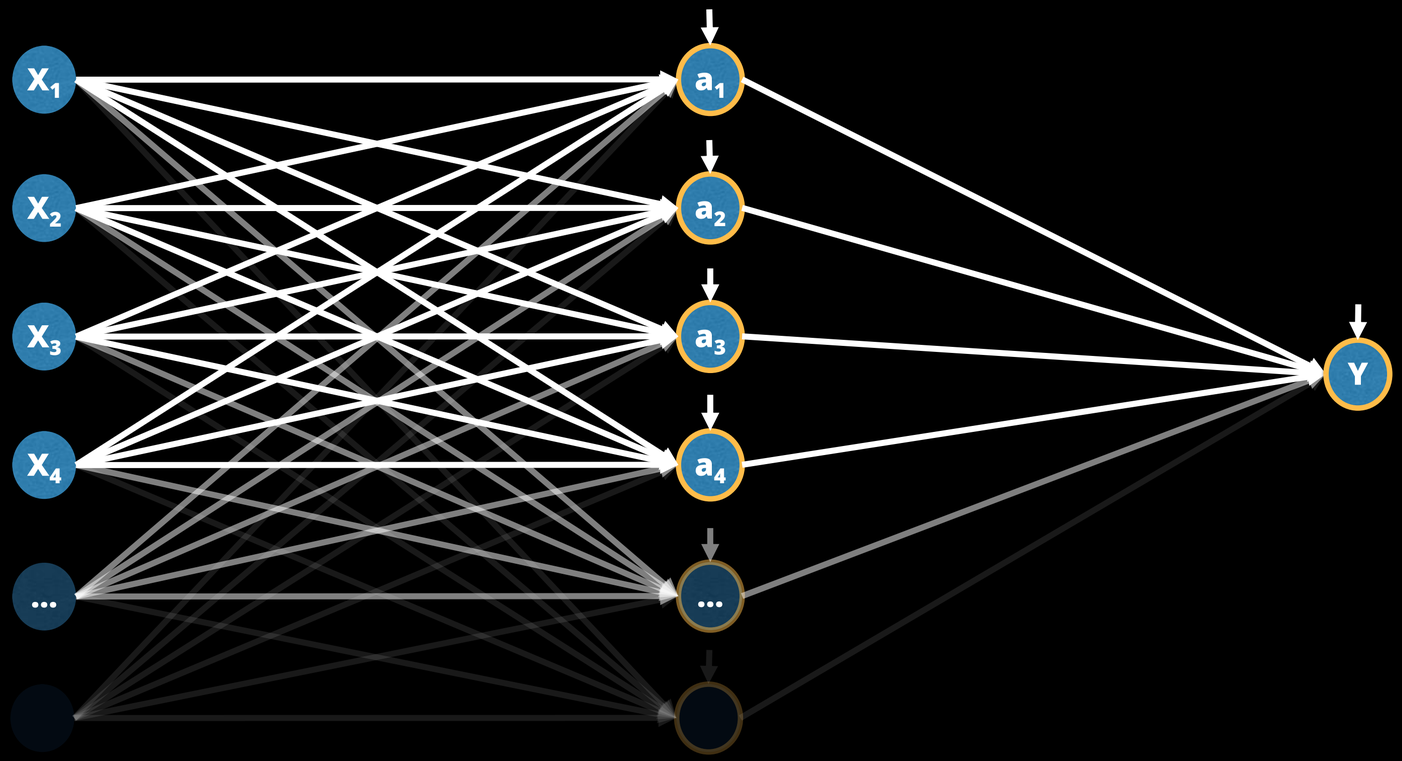
Besteht aus Input-Variablen/ Features/ Dimensionen

Hidden Layer

Fasst mit Hilfe von Aktivierungsfunktionen und geschätzten Gewichten die Werte der vorherigen Schicht in jeweils einem Neuron zusammen.

Output Layer

Fasst ebenfalls mit Hilfe von Aktivierungsfunktion und geschätzten Gewichten die Werte der vorherigen Schicht zusammen.



WICHTIGE KONZEPTE

Forward Propagation:

- **Berechnung der Vorhersage basierend auf den aktuellen Parametern und den definierten Aktivierungsfunktionen**

Backward Propagation:

- **Berechnung des Schätzfehlers mit Hilfe der Kostenfunktion**
- **Berechnung neuer, verbesserter Parameter mit Hilfe der Optimierungsfunktion**

MODELLGÜTEKRITERIEN

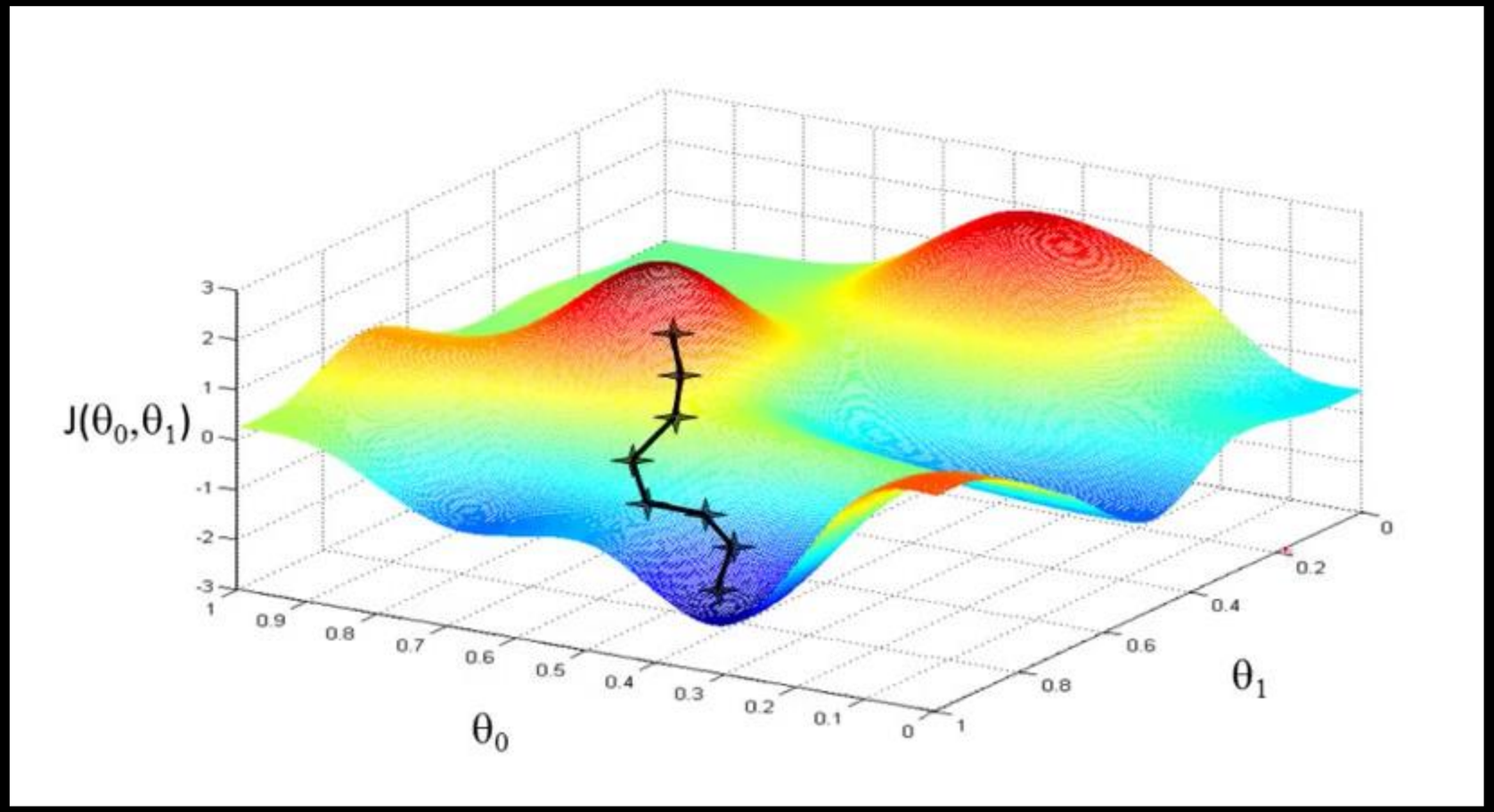
errors:	forecast – actual	(auch: residuals)
mae:	mean(abs(errors))	
mape:	mean(abs(errors/actual))	
mse:	mean(errors^2)	
rmse:	sqrt(mean(errors^2))	
rse:	sum(errors^2) / sum((actual-mean(actual))^2)	
r^2 =	1 – rse	

Video (3 Minuten) mit Erklärung und Darstellung der Kriterien:
https://www.coursera.org/lecture/machine-learning-with-python/evaluation-metrics-in-regression-models-5SxtZ

HYPERPARAMATER IN NEURONALEN NETZEN

- **Wahl der Architektur:**
  - Anzahl der Hidden Layer des Netzes
  - Typen der Hidden Layer
  - Anzahl der Neuronen je Hidden Layer
  - Wahl der Aktivierungsfunktionen
- Wahl der Kostenfunktion („Loss Function“)
- Wahl der Optimierungsfunktion („Optimizer“)
- **Wahl der Parameter des Optimizers**

PARAMETER VON OPTIMIZERN



- **Schrittgröße für die Annäherung an das Kosten-Minimum („Learning Rate“)**
- **Trägheit bei Richtungsänderungen („Momentum“)**

Quelle: https://www.coursera.org/learn/machine-learning

PARAMETER DES OPTIMIZERS „ADAM“

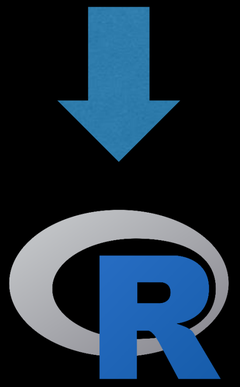
- **Lernparameter/ Schrittweite der Optimierung : alpha (learning rate)**

- **Trägheit der Optimierung: beta1 and beta2 (momentum)**

R VS. PYTHON

Statistische Verfahren außerhalb des ML

Mathematik/ Statistik und Disziplinen mit Anwendungsbereichen von Statistik (Ökonometrie, Psychometrie, Biometrie, ...)



Statistische Verfahren des ML

Angewandte Informatik und freie Wirtschaft mit Anwendungsbereichen von ML



INSTALLATION VON PYTHON

```
---
title: "R Notebook"
output: html_notebook
---

### Installation von Python und der für TensorFlow benötigten Pakete (nur einmalig nö

```{r}
install.packages("reticulate")
library(reticulate)

# Installation von miniconda (falls nicht vorhanden)
install_miniconda(update=TRUE)


# Anlegen einer speziellen Python Umgebung
conda_create("r-reticulate", python_version = "3.8" )

# Installieren der Pakete in der angelegten Umgebung
conda_install("r-reticulate", "pandas")
conda_install("r-reticulate", "numpy")
conda_install("r-reticulate", "tensorflow")
conda_install("r-reticulate", "h5py")

# Verwenden der speziellen Python Umgebung die zuvor erstellt wurde
use_condaenv("r-reticulate")

```
```

VERWENDUNG VON PYTHON IN RSTUDIO

````
```{python}
import sys
import tensorflow

# Augabe der installierten Python- und TensorFlow-Versionen
print("Python Version:      " + sys.version+"\nTensorFlow Version: "+tensorflow.__version__)

```

```{r}
# Import Libraries
library(reticulate)


# Importing Data
data <- mtcars

```


```{python}
mpg = r.data['mpg']

```


```{r}
table(py$mpg)

```
````

Siehe auch: https://rstudio.github.io/reticulate/articles/r_markdown.html

DATENAUFBEREITUNG FÜR TENSORFLOW

```
1
2  ##################################################
3  ### Preparation of the Environment 🛋
20 ##################################################
21 ### Data Import 🛋
27 ##################################################
28 ### Data Preparation ####
29
30 # Recoding of the variables into one-hot encoded (dummy) variables
31 dummy_list <- c("view", "condition")
32 house_pricing_dummy = dummy_cols(house_pricing, dummy_list)
33
34 # Definition of lists for each one-hot encoded variable (just to make the handling easier)
35 condition_dummies = c('condition_1', 'condition_2', 'condition_3', 'condition_4', 'condition_5')
36 view_dummies = c('view_0', 'view_1', 'view_2', 'view_3','view_4')
37
38
39 ##################################################
40 ### Selection of the Feature Variables and the Label Variable ####
41
42 # Selection of the features (the independent variables used to predict the dependent)
43 #features <- c('sqft_lot', 'waterfront', 'grade', 'bathrooms', view_dummies, condition_dummies)
44 features <- c('sqft_lot', 'waterfront', 'grade', 'bathrooms', condition_dummies, view_dummies)
45 # Selection of the label (the dependent variable)
46 labels <- 'price'
47
48
49 ##################################################
50 ### Selection of Training, Validation and Test Data ####
51
52 # Look at the data
53 str(house_pricing_dummy)
54
55 # Setting the random counter to a fixed value, so the random initialization stays the same (the ra
56 set.seed(1)
57
```

BREAKOUT

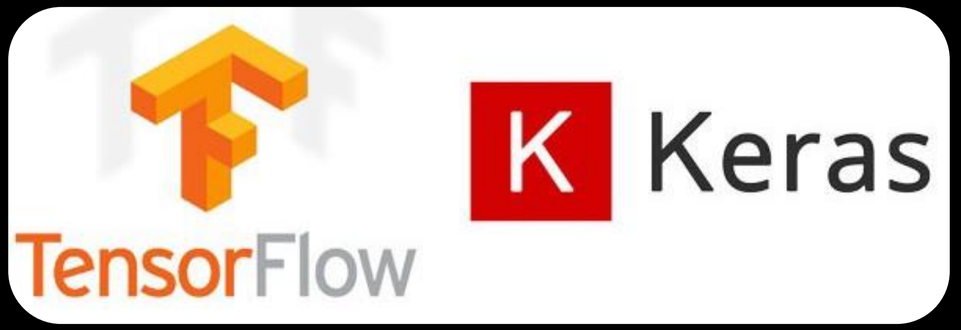
- Schaut Euch zusammen den Code zur Datenaufbereitung an und geht ihn Schritt für Schritt einmal durch.

- Diskutiert, was in den einzelnen Schritten getan wird!


TensorFlow
K
Keras


scikit
learn


PYTORCH



- **Feb 2017: TensorFlow 1.0 (Estimator API)**
- **Nov 2017: TensorFlow 1.4 (Estimator API, Keras API)**
- **Jan 2019: TensorFlow 2.0 (Estimator API, Keras API)**

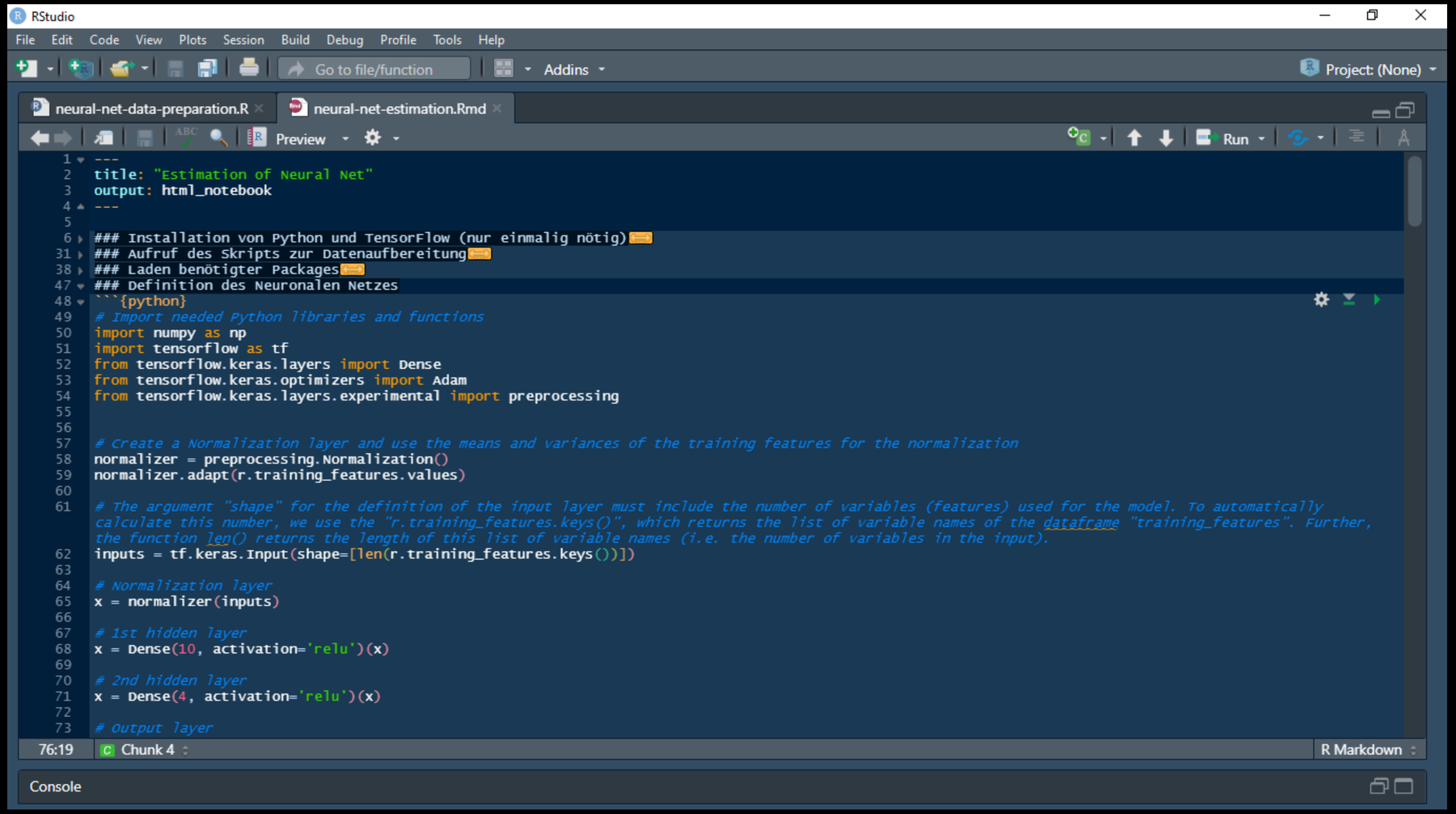
NUTZUNG VON KERAS IN R

Keras ist eine Schnittstelle (API/ ein Funktionswrapper) zur vereinfachenden Nutzung von TensorFlow.

Prinzipiell zwei Varianten :

- **Nutzung des Packages „keras“ (vgl. https://keras.rstudio.com/)**
- **Nutzung von Keras in Python und die Integration von Python über das Paket „reticulate“**

DEFINITION EINES NEURONALEN NETZES



BATCH UND EPOCHE

Batch

- **Eine Menge von Fällen, die genutzt wird, um die Gewichte des Modells zu optimieren.**
- **Optimierungsschritte werden anhand der Batches durchgeführt.**

Epoche

- **Optimierung des Modells anhand aller Fälle bzw. aller Batches zu einem Trainingsdatensatz**

Je nach Modell kann es notwendig sein, es über mehrere hundert oder tausend Epochen zu optimieren.

NORMALISIERUNG

Definition:

- **Abziehen des Mittelwerts und Teilen durch die Standardabweichung.**

Erleichtert es dem neuronalen Netz, die benötigten Parameter zu erlernen, indem alle Werte in einem ähnlichen Wertebereich liegen.

BATCH-NORMALISIERUNG

- Durchführung der Normalisierung auf Batch-Ebene

Zusätzliche Optimierung:

- exakt gleiche Mittelwerte und Standardabweichungen sind nicht notwendigerweise optimal im Sinne der Modellierung

→ Einfügen der Normalisierungsparameter als trainierbare Parameter

AUFGABEN

- Untersucht alle Eure Modellvariablen auf die Existenz von fehlenden und unplausiblen Werten
- Trainiert ein erstes neuronales Netz für Euren Datensatz (Löscht dazu zunächst alle Zeilen mit fehlenden Werten)
- Wenn ihr etwas mehr über Python lernen wollt, könnt Ihr diese Einführung auf Kaggle nutzen.